Ministerul Educaţiei şi Cercetării al Republicii Moldova

Universitatea Tehnică a Moldovei

Facultatea Calculatoare, Informatică şi Microelectronică

Laboratory Work No. 1:

The Caesar Cipher and the Caesar Cipher with a Permutation

Course: Cryptography and Security

Elaborated: st. gr. FAF-241 Lungu Ilie

Verified: asist. univ. *Zaica Maia*

Chişinău – 2026

Contents

1 SHORT THEORY **2**
- 1.1 The Caesar Cipher . . . 2
- 1.2 The Caesar Cipher with a Permutation . . . 2

2 IMPLEMENTATION **3**
- 2.1 Letter Encoding . . . 3
- 2.2 Building the Permuted Alphabet . . . 3
- 2.3 Encryption and Decryption . . . 4
- 2.4 Input Validation and Text Preparation . . . 5
- 2.5 Main Program . . . 6

3 RESULTS **7**
- 3.1 Task 1.1 – Caesar Cipher . . . 7
- 3.2 Task 1.2 – Caesar Cipher with Two Keys . . . 7
- 3.3 Invalid Input . . . 8
- 3.4 Verification Against the Examples of the Assignment . . . 8

4 CONCLUSIONS **8**

5 SOURCE CODE **9**

1 SHORT THEORY

The objective of this laboratory work is to understand how the Caesar cipher works, to see why its small key space makes it insecure, to strengthen it with a keyword-based permutation of the alphabet, and to implement both versions for the Romanian alphabet ($n = 31$ letters).

1.1 The Caesar Cipher

In the Caesar cipher every letter of the plaintext is replaced by the letter found a fixed number of positions further along the alphabet. The secret key k, the same for encryption and decryption, is this shift value: $k \in \{1, 2, \ldots, n-1\}$, where n is the length of the alphabet ($k = 0$ is excluded because it leaves the text unchanged). If x and y denote the numeric codes (positions starting from 0) of a plaintext letter and of a ciphertext letter, the encryption and decryption functions are

$$c = e_k(x) = (x + k) \bmod n, \tag{1}$$

$$m = d_k(y) = (y - k) \bmod n. \tag{2}$$

When $y - k$ is negative the result must be brought back into the range $0 \ldots n - 1$ by adding n; for example $(1 - 3) \bmod 26 = 24$.

Key space. There are only $n - 1$ different keys: 25 for the Latin alphabet and 30 for the Romanian one. A cryptanalyst can therefore try every key in turn (exhaustive search, or brute force); exactly one of the candidates yields readable text in the language of the message. This makes the cipher very weak.

1.2 The Caesar Cipher with a Permutation

To increase the strength of the cipher, the alphabet is first permuted using a second key, the *keyword* k_2 (not to be confused with the numeric key $k_1 = k$). The permuted alphabet is built by writing the letters of k_2 first, each letter only once (only its first occurrence is kept), followed by the remaining letters of the alphabet in their natural order. For the Romanian alphabet and $k_2 =$ CRIPTOGRAFIE the distinct keyword letters are C, R, I, P, T, O, G, A, F, E, and the permuted alphabet is

C R I P T O G A F E Ă Â B D H Î J K L M N Q S Ş Ţ U V W X Y Z.

The Caesar shift is then applied in this new order: the numeric code of a letter is its position in the *permuted* alphabet, and the same formulas are used:

$$c = e_{k_1,k_2}(x) = \pi(x + k_1) \bmod n \text{ in the permuted order}, \tag{3}$$

$$m = d_{k_1,k_2}(y) = \pi(y - k_1) \bmod n \text{ in the permuted order}, \tag{4}$$

where x and y are positions in the permuted alphabet.

Key space. The alphabet can be ordered in $n!$ ways and the shift can take $n - 1$ values, so the number of key pairs is $n!\,(n - 1)$. For the Romanian alphabet

$$31! \times 30 = 246\,685\,159\,625\,337\,684\,531\,766\,886\,400\,000\,000 \approx 2.5 \cdot 10^{35} \approx 2^{117.6},$$

which makes exhaustive search infeasible. However, each plaintext letter is still always replaced by the same ciphertext letter, so the letter frequencies of the language remain visible

and the cipher is vulnerable to *frequency analysis*. Different key pairs may also produce the same substitution, so the number of distinct encryptions never exceeds 31!.

Table 1 summarises both versions.

Table 1: Key space of the two ciphers for the Romanian alphabet ($n = 31$).

Cipher	**Keys**	**Number of keys**
Caesar (Task 1.1)	k	$n - 1 = 30$
Caesar + permutation (Task 1.2)	(k, k_2)	$n!\,(n-1) \approx 2.5 \cdot 10^{35}$

2 IMPLEMENTATION

Both tasks are solved in one Python file, `lab1-2.py`, built around the class `CaesarCipher`. Task 1.1 is the particular case in which no keyword is entered: the working alphabet is then the natural one. When a keyword is entered (Task 1.2), it has to satisfy the extra conditions of that task. The program asks, in this order, for the operation, the key k, the keyword (may be left blank) and the message, and then prints the alphabet used together with the result.

2.1 Letter Encoding

The alphabet is stored as an ordered string of the 31 Romanian letters, and the numeric code of a letter is its position in that string (Table 2 of the assignment: A = 0, Ă = 1, ..., Z = 30). No built-in character encoding (ASCII, `ord()`) is used to compute shifts.

```
ROMANIAN_ALPHABET = 'AĂÂBCDEFGHIÎJKLMNOPQRSTUVWXYZ'

class CaesarCipher:
    def __init__(self, ordered_alphabet: str):
        self.alphabet = list(ordered_alphabet)
        self.N = len(self.alphabet)          # n = 31
        self.k = 0
        self.kw = []
        # no keyword -> the working alphabet is the natural one (
           Task 1.1)
        self.altered_alphabet = list(self.alphabet)
        self.altered_mapping = {l: i for i, l in enumerate(self.
           altered_alphabet)}
```

Listing 1: Alphabet definition and the `CaesarCipher` constructor.

`altered_mapping` is the letter → code dictionary of the working alphabet. Because the class receives the alphabet as a parameter, it works unchanged for any alphabet (it is also used with the Latin one in Section 3 to check the examples of the assignment).

2.2 Building the Permuted Alphabet

The keyword is reduced to its distinct letters in order of first appearance, and the permuted alphabet is the keyword letters followed by all the remaining letters in natural order.

```python
def unique_letters(text: str) -> list[str]:
    # keeps only the first occurrence of every letter, in order
       of appearance
    return list(dict.fromkeys(text))

def set_cipher_params(self, k: int, keyword: str):
    self.k = k
    self.kw = unique_letters(keyword)
    self.set_keyword_params()

def set_keyword_params(self):
    self.altered_alphabet = self.kw + [l for l in self.alphabet
       if l not in self.kw]
    self.altered_mapping = {l: i for i, l in enumerate(self.
       altered_alphabet)}
```

Listing 2: Permuted alphabet: keyword letters first, then the rest.

`dict.fromkeys` preserves insertion order, so repeated letters (the second R, I and A in CRIPTOGRAFIE) are dropped while the order of the first occurrences is kept. A plain `set` must not be used here: its iteration order is arbitrary and would give a different alphabet at each run, making decryption impossible. The program prints `altered_alphabet` so that the result can be checked by hand.

2.3 Encryption and Decryption

Encryption converts every letter to its position in the permuted alphabet, adds k modulo n, and converts the new positions back to letters of the *same* permuted alphabet. Decryption is identical with $-k$.

```python
def shift_indexes(self, indexes: list[int], k: int) -> list[int]:
    return [(i + k) % self.N for i in indexes]

def cipher(self, message: str) -> str:
    message_indexes = [self.altered_mapping[l] for l in message]
    shifted_indexes = self.shift_indexes(message_indexes, self.k)
    return ''.join(self.altered_alphabet[i] for i in
       shifted_indexes)

def decipher(self, cipher: str) -> str:
    cipher_indexes = [self.altered_mapping[l] for l in cipher]
    shifted_indexes = self.shift_indexes(cipher_indexes, -self.k)
    return ''.join(self.altered_alphabet[i] for i in
       shifted_indexes)
```

Listing 3: Encryption and decryption with the shift modulo n.

In Python the `%` operator always returns a value in $0 \dots n-1$ for a positive modulus, even when the left operand is negative, so $(y - k) \bmod n$ needs no explicit correction. With an empty keyword the permuted alphabet equals the natural one and these functions are exactly the Caesar cipher of Task 1.1.

2.4 Input Validation and Text Preparation

Each validation method returns a message stating the allowed values, or `None` when the input is valid.

```
MIN_KEYWORD_LENGTH = 7

def check_k(self, k: int):
    if not 1 <= k <= self.N - 1:
        return f'Invalid key k = {k}. The key must be an integer between 1 and {self.N - 1} inclusive.'
    return None

def check_text(self, text: str):
    for position, letter in enumerate(text, start=1):
        if letter not in self.alphabet:
            return (f"Invalid character '{letter}' at position {position}. "
                    f"Only letters of the Romanian alphabet (A-Z, Ă, Â, Î, Ș, Ț, "
                    f"uppercase or lowercase) and spaces are allowed.")
    if not text:
        return 'The text is empty. Enter at least one letter.'
    return None

def check_keyword(self, keyword: str):
    if keyword == '':
        return None          # no keyword -> plain Caesar cipher (Task 1.1)
    for position, letter in enumerate(keyword, start=1):
        if letter not in self.alphabet:
            return (f"Invalid character '{letter}' at position {position} of the keyword. "
                    f"The keyword may contain only letters of the Romanian alphabet.")
    if len(keyword) < MIN_KEYWORD_LENGTH:
        return (f'The keyword is too short ({len(keyword)} characters). '
                f'It must have at least {MIN_KEYWORD_LENGTH} characters.')
    return None
```

Listing 4: Validation of the key, the text and the keyword.

The key must lie in $1 \ldots 30$ (the upper bound is $n - 1$, so $k = 31$ is rejected: it would be equivalent to $k = 0$). For text and keyword the message names the rejected character and its position. The keyword must have at least 7 characters.

Before validation, text and keyword are converted to uppercase and spaces are removed. The cedilla variants Ş and Ţ typed on some keyboards are mapped to the comma-below letters (S and T with a comma below) required by the alphabet, as recommended in the assignment.

```
CEDILLA_TO_COMMA = str.maketrans({'\u015e': '\u0218', '\u0162': '\u021a'})
```

```
2
3  def format_text(text: str) -> str:
4      return text.upper().translate(CEDILLA_TO_COMMA).replace(' ',
           '')
5
6  def ask(prompt: str, convert, check):
7      """Asks for a value until it is valid; prints the reason on
           every failure."""
8      while True:
9          raw = input(prompt)
10         try:
11             value = convert(raw)
12         except ValueError:
13             print(f"Invalid input '{raw}'. An integer is expected
                   .")
14             continue
15         error = check(value)
16         if error is None:
17             return value
18         print(error)
```

Listing 5: Text preparation and the re-prompting input loop.

The function `ask` repeats the question until the value passes the check, which satisfies the requirement to display a clear message and ask for the value again. A non-numeric key is caught by the `ValueError` branch.

2.5 Main Program

```
1  if __name__ == "__main__":
2      cipherer = CaesarCipher(ROMANIAN_ALPHABET)
3      input_option = ask('Enter option (1 - encryption, 2 -
           decryption): ', int,
4                         lambda o: None if o in (1, 2) else '
                              Invalid option. Enter 1 or 2.')
5      input_k = ask(f'Enter key k (1-{cipherer.N - 1}): ', int,
           cipherer.check_k)
6      input_kw = ask(f'Enter keyword (at least {MIN_KEYWORD_LENGTH}
           letters, blank if not used): ',
7                     format_text, cipherer.check_keyword)
8      input_text = ask('Enter message: ', format_text, cipherer.
           check_text)
9
10     cipherer.set_cipher_params(input_k, input_kw)
11     print('Alphabet used:', ' '.join(cipherer.altered_alphabet))
12     if input_option == 1:
13         print('Ciphertext:', cipherer.cipher(input_text))
14     else:
15         print('Plaintext: ', cipherer.decipher(input_text))
```

Listing 6: Main program flow.

3 RESULTS

All outputs below were produced by running `lab1-2.py`; the user input appears after each prompt.

3.1 Task 1.1 – Caesar Cipher

The message *"Ana are mere"* is encrypted with $k = 5$ and no keyword (Listing 7), then the ciphertext is decrypted back (Listing 8). By hand, with the codes of Table 2: A $(0) \to 5 =$ D, N $(16) \to 21 =$ S, R $(20) \to 25 =$ U, E $(6) \to 11 =$ Î, M $(15) \to 20 =$ R, which agrees with the program.

```
Enter option (1 - encryption, 2 - decryption): 1
Enter key k (1-30): 5
Enter keyword (at least 7 letters, blank if not used):
Enter message: Ana are mere
Alphabet used: A  Â B C D E F G H I Î J K L M N O P Q R S  T  U V W X Y Z
Ciphertext: DSDDUÎRÎUÎ
```

Listing 7: Task 1.1 – encryption ($k = 5$, no keyword).

```
Enter option (1 - encryption, 2 - decryption): 2
Enter key k (1-30): 5
Enter keyword (at least 7 letters, blank if not used):
Enter message: DSDDUÎRÎUÎ
Alphabet used: A  Â B C D E F G H I Î J K L M N O P Q R S  T  U V W X Y Z
Plaintext:  ANAAREMERE
```

Listing 8: Task 1.1 – decryption of the previous ciphertext.

3.2 Task 1.2 – Caesar Cipher with Two Keys

The message *"Ştefan cel Mare"* is encrypted with $k_1 = 7$ and $k_2 =$ criptografie (Listing 9); the program displays the permuted alphabet. By hand: Ş has position 23 in the permuted alphabet and $(23 + 7) \bmod 31 = 30$, the letter Z; T has position 4 and $4 + 7 = 11$, the letter Â. Both match the output. Listing 10 decrypts the result.

```
Enter option (1 - encryption, 2 - decryption): 1
Enter key k (1-30): 7
Enter keyword (at least 7 letters, blank if not used): criptografie
Enter message: tefan cel Mare
Alphabet used: C R I P T O G A F E  Â B D H Î J K L M N Q S  U V W X Y Z
Ciphertext: ZÂJÎHWAJUVHFJ
```

Listing 9: Task 1.2 – encryption ($k_1 = 7$, $k_2 =$ `criptografie`).

```
Enter option (1 - encryption, 2 - decryption): 2
Enter key k (1-30): 7
Enter keyword (at least 7 letters, blank if not used): criptografie
Enter message: ZÂJÎHWAJUVHFJ
Alphabet used: C R I P T O G A F E  Â B D H Î J K L M N Q S  U V W X Y Z
Plaintext:  TEFANCELMARE
```

Listing 10: Task 1.2 – decryption of the previous ciphertext.

3.3 Invalid Input

Listing 11 shows the messages displayed for a key that is too small (0), too large (31) or not a number, for keywords that are too short or contain a non-letter, and for a message with an invalid character. After every error the program asks for the value again. The last line shows that the text typed with a cedilla (ştefan) is accepted.

```
Enter option (1 - encryption, 2 - decryption): 1
Enter key k (1-30): 0
Invalid key k = 0. The key must be an integer between 1 and 30 inclusive.
Enter key k (1-30): 31
Invalid key k = 31. The key must be an integer between 1 and 30 inclusive.
Enter key k (1-30): abc
Invalid input 'abc'. An integer is expected.
Enter key k (1-30): 5
Enter keyword (at least 7 letters, blank if not used): cheie
The keyword is too short (5 characters). It must have at least 7 characters.
Enter keyword (at least 7 letters, blank if not used): cheie1234
Invalid character '1' at position 6 of the keyword. The keyword may contain only
    letters of the Romanian alphabet.
Enter keyword (at least 7 letters, blank if not used): criptografie
Enter message: Hello 7 world
Invalid character '7' at position 6. Only letters of the Romanian alphabet (A-Z, , Â
    , Î, , , uppercase or lowercase) and spaces are allowed.
Enter message: tefan
Alphabet used: C R I P T O G A F E  Â B D H Î J K L M N Q S   U V W X Y Z
Ciphertext: XEHDBU
```

Listing 11: Messages displayed for invalid input.

3.4 Verification Against the Examples of the Assignment

Since the class takes the alphabet as a parameter, it was also run with the Latin alphabet to reproduce the worked examples given in the assignment: with $k = 3$ the plaintext CIFRULCEZAR gives FLIUXOFHCDU; with the additional keyword CRYPTOGRAPHY the permuted alphabet is `CRYPTOGAHBDEFIJKLMNQSUVWXZ` and the ciphertext is PLKTXQPJYDT; and the brute-force example (ciphertext SILKVWFITVRKKRTB, $k = 17$) decrypts to BRUTEFORCEATTACK. All three results coincide with the values in the assignment, and a randomized test of 2 000 random keys and messages over the Romanian alphabet confirmed that `decipher(cipher(m))` returns m in every case.

4 CONCLUSIONS

In this laboratory work the Caesar cipher and its keyword-permutation variant were studied and implemented for the Romanian alphabet of 31 letters. The plain Caesar cipher has only 30 possible keys, so an attacker can try all of them in a fraction of a second and pick the one that gives readable Romanian text; it offers no real protection.

Permuting the alphabet with a keyword raises the number of key pairs to $31! \times 30 \approx 2.5 \cdot 10^{35}$, which rules out exhaustive search. Nevertheless the cipher remains a monoalphabetic substitution: every plaintext letter is always replaced by the same ciphertext letter, so the letter frequencies of Romanian stay visible in the ciphertext and frequency analysis can recover the message without testing the keys. The larger key space therefore increases the strength against brute force but does not make the cipher secure.

From the implementation side, several practical points were learned: the shift must be computed on positions in the (permuted) alphabet and the result must be read from the same alphabet; the permuted alphabet must be built in a deterministic order (a `set` would break decryption); the key range is $1 \dots n-1$, not $1 \dots n$; and the input must be validated with clear messages, including the normalization of the cedilla variants of Ş and Ţ, so that text typed on different keyboards is accepted.

5 SOURCE CODE

The full source code is available in the GitHub repository:
https://github.com/code-bugg/criptografic